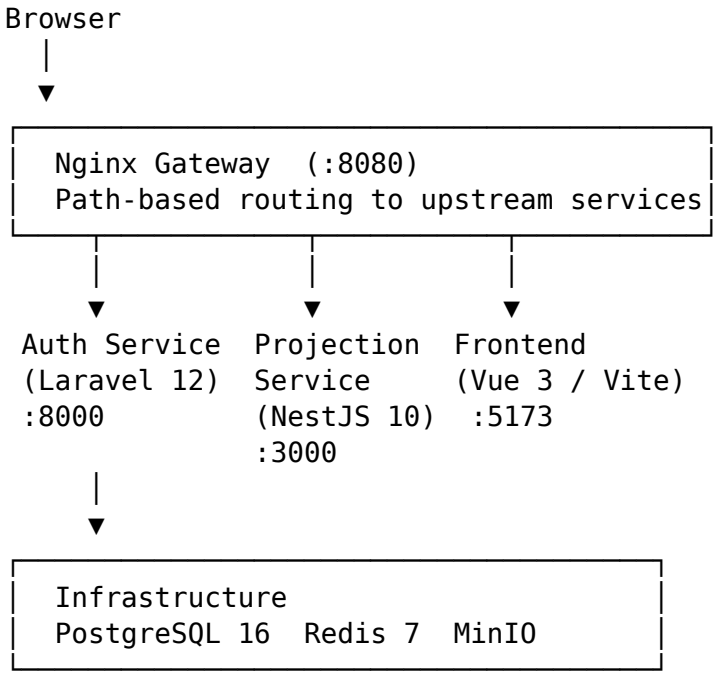
Contents

SaintAugustin – Architecture Overview

SaintAugustin is a church music management platform that helps worship teams organise songs, build service playlists, and project lyrics live to a screen. The project is structured as a monorepo containing two backend services, a Vue 3 frontend, and shared infrastructure defined in Docker Compose.

High-Level Architecture



All HTTP traffic from the browser passes through the Nginx gateway on port 8080. The gateway routes by path prefix:

Path prefix	Upstream
`/api/auth/`	Auth Service
`/api/songs`	Auth Service

Path prefix	Upstream
`/api/songbooks`	Auth Service
`/api/playlists`	Auth Service
`/api/share/`	Auth Service
`/api/sheets`	Auth Service
`/api/projection/`	Projection Service
`/ws/projection/`	Projection Service (WebSocket)
`/socket.io/`	Projection Service (Socket.IO)
`/`	Frontend

Services

Service	Language / Framework	Database	Storage
Auth Service	PHP 8.3 / Laravel 12	PostgreSQL (`sa_auth`)	MinIO (song sheets)
Projection Service	TypeScript / NestJS 10	Redis 7 (session state)	—
Frontend	TypeScript / Vue 3 + Vite	—	—

Deliberately merged services

Several logically distinct capabilities were merged into the Auth Service to reduce operational overhead during development:

- **Song & Songbook CRUD** (would be a separate Song Service in production)
- **File Service** (song sheet upload/download via MinIO presigned URLs)
- **Playlist & Share Link management** (would be a separate Playlist Service)

The Nginx config and Docker Compose retain stub upstreams (`song-service`, `playlist-service`, `file-service`) so these can be extracted into their own containers later without changing the gateway configuration.

User Roles

Role	Capabilities
`admin`	Full CRUD on all entities; manages invitations and songbooks
`musician`	Can create and edit songs; upload song sheets; read-only on songbooks

Role	Capabilities
`projectionist`	Read access; can create and control projection sessions

Registration is invitation-only. Admins issue invitations that carry a pre-assigned role.

Development Phases

Phase	Focus	Status
0	Scaffolding, Docker Compose, CI pipeline	Done
1	Auth, Songs, Songbooks, frontend shell	Done
2	Musician view, ChordPro parser, song sheets (MinIO)	Done
3	Playlists, Playlist Builder, share links, PDF export	Done
4	Live projection (WebSocket sessions, slide renderer)	In progress
5+	Import service, mobile optimisation, offline mode	Planned

Repository Layout

```
saint-augustin/
├── docker/
│   ├── nginx/default.conf      Nginx gateway config
│   └── postgres/init-databases.sh  Creates sa_auth DB on first start
├── docs/                       This documentation folder
├── frontend/                   Vue 3 + Vite + Tailwind SPA
│   └── src/
│       ├── api/                Axios API clients per resource
│       ├── components/         Shared UI components
│     ├── lib/           Pure logic: ChordPro parser, projection slide builder
│       ├── router/             Vue Router with auth guards
│       ├── stores/             Pinia stores
│       ├── types.ts            Shared domain types
│       └── views/              One component per route
```

```
├── services/
│   ├── auth/              Laravel 12 monolith (auth + songs + playlists + files)
│   └── projection/                NestJS 10 WebSocket service
├── docker-compose.yml
└── Makefile
```

Auth Service

The Auth Service is a Laravel 12 application that acts as the primary backend for SaintAugustin. It handles authentication, user management, songs, songbooks, song sheets, playlists, and share links — capabilities that will be split into separate microservices as the project scales.

- **Runtime:** PHP 8.3 with Laravel Octane (FrankenPHP worker)
- **Database:** PostgreSQL 16 (`sa_auth` schema)
- **Object storage:** MinIO (S3-compatible) for song sheet files
- **Auth mechanism:** Laravel Sanctum — stateless Bearer tokens
- **Container port:** 8000 (exposed via Nginx gateway at `/api/...`)

API Routes

All routes are prefixed `/api` by Laravel. The Nginx gateway routes them from port 8080.

Authentication (`/api/auth`)

Method	Path	Auth	Description
GET	`/api/auth/status`	—	Service health / version
POST	`/api/auth/login`	—	Obtain a Bearer token
POST	`/api/auth/password/forgot`	—	Send reset email
POST	`/api/auth/password/reset`	—	Apply reset token
GET	`/api/auth/invitations/{token}`	—	Verify an invitation token
POST	`/api/auth/register`	—	Register via invitation
POST	`/api/auth/logout`	Bearer	Revoke current token
POST	`/api/auth/refresh`	Bearer	Issue a fresh token
POST	`/api/auth/invitations`	Bearer + Admin	Create an invitation

Registration is invitation-only. An admin calls `POST /api/auth/invitations` with `{ email, roles[] }`, which sends an email containing a single-use token. The recipient visits the registration form, the token is validated, and their account is created with the pre-assigned roles.

Songbooks (`/api/songbooks`)

Method	Path	Auth	Description
GET	`/api/songbooks`	Bearer	List all songbooks
GET	`/api/songbooks/{id}`	Bearer	Get a single songbook
POST	`/api/songbooks`	Bearer + Admin	Create a songbook
PUT	`/api/songbooks/{id}`	Bearer + Admin	Update a songbook
DELETE	`/api/songbooks/{id}`	Bearer + Admin	Delete a songbook

Songbooks are organisational containers for songs (e.g. “Hillsong”, “Hymns”, “Originals”). Every song belongs to one songbook.

Songs (`/api/songs`)

Method	Path	Auth	Description
GET	`/api/songs`	Bearer	Paginated list with filtering
GET	`/api/songs/{id}`	Bearer	Get a single song
POST	`/api/songs`	Bearer (Admin/Musician)	Create a song
PUT	`/api/songs/{id}`	Bearer (Admin/Musician)	Update a song
DELETE	`/api/songs/{id}`	Bearer + Admin	Soft-delete
POST	`/api/songs/{id}/restore`	Bearer + Admin	Restore from trash
GET	`/api/songs/{songId}/sheets`	Bearer	List sheet attachments
POST	`/api/songs/{songId}/sheets`	Bearer (Admin/Musician)	Upload a sheet

Song list query parameters: `q` (search title/author), `songbook`, `key`, `tag`, `trashed` (`with`/`only`), `per_page`, `page`.

Songs store ChordPro lyrics in the `lyrics` column. The `original_key`, `tempo`, `time_signature`, `tags[]`, `ccli_number`, and `preview_url` fields are optional metadata. Songs support soft-delete (via Laravel’s `SoftDeletes`) so the library never loses data.

Song Sheets (`/api/sheets`)

Method	Path	Auth	Description
GET	`/api/sheets/{id}`	Bearer	Get metadata + presigned URL
DELETE	`/api/sheets/{id}`	Bearer + Admin	Delete sheet + file

Song sheets are PDF or image attachments stored in MinIO. On upload the file is persisted to the `saintaugustin` bucket and a record is saved in the `song_sheets` table. On

download the API generates a short-lived presigned URL (configurable TTL, default 15 minutes via `SONG_SHEET_URL_TTL`). If the URL has expired the client refetches `GET /sheets/{id}` to obtain a fresh one.

Playlists (`/api/playlists`)

Method	Path	Auth	Description
GET	`/api/playlists`	Bearer	Paginated list
POST	`/api/playlists`	Bearer	Create a playlist
GET	`/api/playlists/{id}`	Bearer	Get playlist with items
PUT	`/api/playlists/{id}`	Bearer (Owner/Admin)	Update playlist metadata
DELETE	`/api/playlists/{id}`	Bearer (Owner/Admin)	Delete playlist
POST	`/api/playlists/{id}/duplicate`	Bearer	Duplicate (caller becomes owner)
POST	`/api/playlists/{playlistId}/items`	Bearer (Owner/Admin)	Add a song to the playlist
PUT	`/api/playlists/{playlistId}/items/reorder`	Bearer (Owner/Admin)	Reorder items
PUT	`/api/playlists/{playlistId}/items/{itemId}`	Bearer (Owner/Admin)	Update item (key, notes)
DELETE	`/api/playlists/{playlistId}/items/{itemId}`	Bearer (Owner/Admin)	Remove item
GET	`/api/playlists/{playlistId}/share`	Bearer (Owner/Admin)	List share links
POST	`/api/playlists/{playlistId}/share`	Bearer (Owner/Admin)	Create a share link
DELETE	`/api/share-links/{id}`	Bearer (Owner/Admin)	Revoke a share link
GET	`/api/playlists/{id}/export`	Bearer	Download PDF export
GET	`/api/share/{token}`	— (throttled 60/min)	Resolve public share link

Each playlist item stores `position`, an optional `target_key` (transpose destination for the musician), and free-text `notes`. The `reorder` endpoint accepts an ordered array of item IDs and re-assigns `position` values in one transaction.

Share links carry a `mode` field (`musician` or `projection`). The `musician` payload includes full song lyrics; `projection` omits them. Public access is throttled to prevent token enumeration.

PDF export renders the playlist via a Blade view and returns a downloadable PDF using a Laravel PDF package.

Key Design Decisions

- **Sanctum stateless mode only.** The `statefulApi()` helper is not used on any route — all API consumers send `Authorization: Bearer {token}` headers. This avoids CSRF complications in the SPA and keeps the service truly stateless.
- **Soft deletes on songs.** Songs are never hard-deleted by users. The `trashed` query parameter on the list endpoint lets admins view and restore deleted songs.
- **MinIO presigned URLs.** File content never passes through the Laravel process on download; only the presigned URL is issued. This keeps the auth service lean and avoids large request bodies in PHP.
- **Single-service monolith for phases 1–3.** Songs, sheets, and playlists live in the same Laravel app to simplify development. The Nginx gateway already defines stub upstreams for future extraction.

Environment Variables

Variable	Description	Default
`APP_KEY`	Laravel app encryption key	— (generate with `php artisan key:generate`)
`DB_HOST` / `DB_DATABASE` / `DB_USERNAME` / `DB_PASSWORD`	PostgreSQL connection	—
`REDIS_HOST` / `REDIS_PORT`	Redis connection (cache + queue)	`redis:6379`
`JWT_SECRET`	Sanctum token signing secret	—
`MINIO_ENDPOINT`	MinIO S3 endpoint	`http://minio:9000`
`MINIO_ACCESS_KEY` / `MINIO_SECRET_KEY`	MinIO credentials	—
`MINIO_BUCKET`	Default bucket name	`saintaugustin`
`SONG_SHEET_URL_TTL`	Presigned URL lifetime (minutes)	`15`

Projection Service

The Projection Service provides real-time slide control for live worship services. A worship leader (controller) drives a playlist of lyrics slides from their device; one or more projection displays (screens, secondary browsers) follow along via WebSocket.

- **Runtime:** Node.js / NestJS 10 with Socket.IO
- **Session storage:** Redis 7 (ephemeral — sessions expire automatically)
- **Container port:** 3000
- **Gateway paths:** `/api/projection/` (HTTP), `/ws/projection/` and `/socket.io/` (WebSocket)

Concepts

Session

A **projection session** is a short-lived object stored in Redis that holds:

Field	Type	Description
`id`	UUID	Unique session identifier
`playlistId`	string | null	Source playlist (if launched from one)
`playlistName`	string	Display name
`slides`	`ProjectionSlide[]`	Ordered flat list of all slides
`currentIndex`	number	Index of the active slide
`blackout`	boolean	When true, displays show a blank screen
`fontScale`	number	Font size multiplier (0.5–3.0)
`createdAt` / `updatedAt`	ISO strings	Timestamps

Sessions have a configurable TTL (default 4 hours) that resets on every mutation. A session that is not being actively controlled will expire automatically, cleaning up Redis without any explicit teardown.

Slides

Slides are built from playlist items **on the frontend** before the session is created. Each slide maps to one lyric section of one song:

```
interface ProjectionSlide {
  id: string          // unique per slide
  itemIndex: number   // which playlist item this slide belongs to
  slideIndex: number  // position within that item's slides
  songTitle: string
  section: string | null  // e.g. "Verse 1", "Chorus"
  body: string            // plain text lyrics for this section
}
```

The slide-building logic lives in `frontend/src/lib/projection/buildSlides.ts`. It splits ChordPro lyrics into sections and creates one `ProjectionSlide` per section.

Control Token

When a session is created the service returns a `controlToken` — a cryptographically random 24-byte base64url string. This token is stored in Redis under a separate key (`sa:proj:control-token:{sessionId}`) so it never appears in the broadcast state. Only the client that holds the control token can issue write events (next, previous, goto, blackout, font-scale). Token comparison uses a constant-time algorithm to avoid timing oracle attacks.

HTTP API

Method	Path	Auth	Description
POST	`/sessions`	—	Create a session; returns `{ sessionId, controlToken, state }`
GET	`/sessions/:id`	—	Fetch current session state (used on initial display page load)
DELETE	`/sessions/:id`	`X-Control-Token` header	End the session
GET	`/health`	—	Service health check

Session creation accepts:

```
{
  "playlistName": "Sunday 27 April",
  "playlistId": "uuid-optional",
  "slides": [ ... ]
}
```

The HTTP API is intentionally unauthenticated at the user level. The service runs inside the church's private network and sessions are short-lived. The `sessionId` (UUID) acts as the read credential; the `controlToken` gates writes.

WebSocket API

Namespace: `/ws/projection`

Connect to `ws://<host>/ws/projection` using Socket.IO. After connecting, emit a `join` event to enter a session room.

Client → Server events

Event	Payload	Role required	Description
`join`	`{ sessionId, controlToken? }`	—	Enter a session room. If `controlToken` matches the stored token the socket is promoted to controller.
`next`	—	controller	Advance to the next slide
`previous`	—	controller	Go back one slide
`goto`	`{ index: number }`	controller	Jump to an absolute slide index

Event	Payload	Role required	Description
`jump-to-song`	`{ itemIndex: number }`	controller	Jump to the first slide of a playlist item
`blackout`	`{ on: boolean }`	controller	Toggle screen blackout
`font-scale`	`{ scale: number }`	controller	Set font scale (clamped 0.5–3.0)

The `join` response includes the current `state` so a newly connected display is immediately in sync. Write events return `{ ok: true }` or `{ ok: false, error: string }`.

Server → Client events

Event	Payload	Description
`state`	`SessionState`	Broadcast to every client in the room after any mutation

The full state is sent on every change rather than a diff. This makes reconnection trivial — a client that drops and reconnects simply re-joins and receives the current state immediately.

Redis Storage Layout

```
sa:proj:session:<sessionId>     JSON-encoded SessionState (TTL: SESSION_TTL_SECONDS)
sa:proj:control-token:<sessionId>  controlToken string        (same TTL)
```

Keeping the control token in a sibling key (rather than inside the state object) ensures it is never accidentally included in a WebSocket broadcast.

Every mutation refreshes both keys' TTL atomically using a Redis pipeline.

Module Structure

```
src/
├── app.module.ts              Root NestJS module
├── main.ts              Bootstrap (Socket.IO adapter, CORS, validation pipe)
├── health.controller.ts      GET /health
├── projection/
│   ├── projection.module.ts
│   └── projection.gateway.ts WebSocket gateway (Socket.IO)
├── redis/
│   ├── redis.module.ts
```

```
│   └── redis.service.ts      Thin ioredis wrapper
└── sessions/
    ├── sessions.module.ts
    ├── sessions.controller.ts HTTP CRUD for sessions
    ├── sessions.service.ts    Session lifecycle + mutations
    ├── session.types.ts       SessionState interface
    └── dto/
        ├── create-session.dto.ts
        └── slide.dto.ts
```

Environment Variables

Variable	Description	Default
`PORT`	HTTP + WebSocket port	`3000`
`REDIS_HOST`	Redis hostname	`redis`
`REDIS_PORT`	Redis port	`6379`
`SESSION_TTL_SECONDS`	Session expiry (seconds)	`14400` (4h)

Frontend

The frontend is a Vue 3 single-page application served by Vite. It communicates with the Auth Service via a REST API and with the Projection Service via Socket.IO WebSocket.

- **Framework:** Vue 3 (Composition API)
- **Build tool:** Vite
- **Styling:** Tailwind CSS + design tokens in `src/assets/tokens.css`
- **State management:** Pinia
- **Routing:** Vue Router 4
- **Testing:** Vitest + Vue Test Utils

Routing

Routes are defined in `src/router/index.ts`. Three meta flags control access:

Meta flag	Effect
`requiresAuth`	Unauthenticated users are redirected to `/login` (attempted path preserved in `?redirect=`)
`requiresAdmin`	Non-admin users are sent to `/`
`requiresEditor`	Users without `admin` or `musician` role are sent to `/`

Meta flag	Effect
`public: true` + `hideForAuthed: true`	Authenticated users are sent to `/` (login, register pages)
`public: true` (no `hideForAuthed`)	Publicly accessible with no auth check (share, projection display)

Route Map

Path	View	Access
`/login`	`LoginView`	Public, hidden for authed
`/register`	`RegisterView`	Public, hidden for authed
`/forgot-password`	`ForgotPasswordView`	Public, hidden for authed
`/reset-password`	`ResetPasswordView`	Public, hidden for authed
`/`	`DashboardView`	Auth required
`/library`	`SongLibraryView`	Auth required
`/songs/new`	`SongEditorView`	Auth + editor role
`/songs/:id`	`SongEditorView`	Auth + editor role
`/songs/:id/play`	`MusicianView`	Auth required
`/admin/songbooks`	`SongbookAdminView`	Auth + admin
`/playlists`	`PlaylistsListView`	Auth required
`/playlists/:id`	`PlaylistBuilderView`	Auth required
`/s/:token`	`SharedPlaylistView`	Public
`/projection/control/:id`	`ProjectionControlView`	Auth required
`/projection/display/:id`	`ProjectionDisplayView`	Public

Pinia Stores

Store	File	Responsibility
`auth`	`stores/auth.ts`	User session, token persistence, role helpers (`isAdmin`, `canEditSongs`)
`songs`	`stores/songs.ts`	Song list, pagination, CRUD actions
`songbooks`	`stores/songbooks.ts`	Songbook list and admin actions
`songSheets`	`stores/songSheets.ts`	Sheet upload, list, and presigned URL management
`playlists`	`stores/playlists.ts`	Playlist CRUD, item management, share links, export
`projection`	`stores/projection.ts`	Projection session lifecycle, WebSocket state sync

The `auth` store persists the Bearer token to `localStorage` so sessions survive page reloads. All other stores are in-memory only.

API Layer

All HTTP calls go through `src/api/client.ts`, which is an Axios instance pre-configured with the base URL (`VITE_API_BASE_URL`) and an interceptor that attaches the `Authorization: Bearer {token}` header from the auth store on every request.

Resource modules:

Module	Endpoint group
`api/auth.ts`	`/api/auth/*`
`api/songs.ts`	`/api/songs`
`api/songbooks.ts`	`/api/songbooks`
`api/songSheets.ts`	`/api/songs/:id/sheets`, `/api/sheets/:id`
`api/playlists.ts`	`/api/playlists`
`api/shareLinks.ts`	Share link sub-resources
`api/invitations.ts`	`/api/auth/invitations`
`api/password.ts`	`/api/auth/password/*`
`api/projection.ts`	`/api/projection/sessions`

ChordPro Library (`src/lib/chordpro`)

A pure-TypeScript ChordPro parser and transposer used in the musician view and slide builder.

Parser (`parser.ts`)

Parses ChordPro text into a structured token stream:

- **Directives** — `{title:}`, `{key:}`, `{start_of_chorus}`, `{soc}`, etc.
- **Chord tokens** — `[Am]`, `[G/B]` inline with lyrics
- **Lyric lines** — plain text between chords

The output is a list of `Section` objects, each containing `Line[]` of `Token[]`.

Transpose (`transpose.ts`)

Shifts all chord tokens from one key to another. Handles:

- Major and minor keys
- Sharps vs flats (follows the target key's convention)
- Slash chords (e.g. `G/B` → `A/C#` when transposing up a tone)

The twelve chromatic steps are represented as two parallel arrays (sharps / flats) and the transpose function picks the correct spelling based on the target key signature.

Projection Library (`src/lib/projection`)

Utilities for converting a playlist into a flat slide list for the Projection Service.

`buildSlides.ts`

Takes a `Playlist` with items and returns `ProjectionSlide[]`. For each item it:

1. Parses the song's ChordPro lyrics.
2. Optionally transposes to the item's `target_key`.
3. Splits sections into individual slides, stripping chord tokens to leave clean lyrics.

`slides.ts`

Helper types and pure functions for working with the slide array (e.g. finding which item a given slide index belongs to).

Key Views

`MusicianView`

Read-only song view for performers. Displays ChordPro lyrics with chords rendered above the corresponding syllables. Includes:

- Live transpose (key selector adjusts rendering without modifying stored data)
- Metronome with visual beat indicator and tap-tempo
- PreviewPlayer for audio preview URLs
- Sheet viewer (PDF/image) via MinIO presigned URLs

`PlaylistBuilderView`

Full-featured playlist editor. Features:

- Drag-and-drop song ordering (or reorder via API)
- Per-item key and notes editor
- Inline song preview
- Share link management (create musician/projection links, revoke, copy URL)
- "Go Live" button that builds slides and creates a projection session

`ProjectionControlView`

Worship leader control panel (requires auth). Connects to the Projection Service WebSocket as a controller using the `controlToken`. Provides:

- Next / Previous slide buttons
- Song jump list (jump directly to the first slide of any playlist item)
- Blackout toggle
- Font scale slider
- Live slide preview

`ProjectionDisplayView`

Public projector screen (no auth, no navigation chrome). Connects as a display subscriber and renders the current slide full-screen. Handles:

- Automatic reconnection on WebSocket drop
- Blackout (blank screen)
- Font scale from session state
- QR / URL display when no session is active

`SharedPlaylistView`

Public read-only playlist view accessed via a share link token. The mode (`musician` or `projection`) is determined by the share link. In musician mode the full song lyrics are shown; in projection mode only the setlist overview is visible.

Components

Component	Description
`AppShell`	Main layout: navigation sidebar, top bar, mobile hamburger
`ChordProPreview`	Renders ChordPro text with chord-above-lyric layout
`SlideRenderer`	Full-screen slide display for projection
`SheetViewer`	PDF/image viewer with presigned URL refresh logic
`ShareDialog`	Modal for creating and managing share links
`KeyBadge`	Pill showing a musical key (colour-coded by accidental type)
`Metronome`	Visual + audio metronome with BPM input and tap-tempo
`PreviewPlayer`	Minimal audio player for preview URLs
`Toast`	Notification system
`Icon`	Wrapper around heroicons SVGs
`BrandMark`	Logo component

Testing

Tests use Vitest with `@vue/test-utils` for component tests. Run all tests:

```
cd frontend
npm test
```

Test files live alongside the code they test in `__tests__` subdirectories.

Environment Variables

Variable	Description
`VITE_API_BASE_URL`	REST API base URL (e.g. `http://localhost:8080/api`)
`VITE_WS_URL`	WebSocket base URL (e.g. `ws://localhost:8080`)

Infrastructure

SaintAugustin's local development environment is fully containerised via Docker Compose. Every service, database, and piece of infrastructure runs as a container; no dependencies need to be installed on the host beyond Docker Engine and Compose v2.

Services at a Glance

Container	Image	Ports	Data
`sa-postgres`	`postgres:16-alpine`	5432	`postgres_data` volume
`sa-redis`	`redis:7-alpine`	6379	`redis_data` volume
`sa-minio`	`minio/minio:latest`	9000 (S3), 9001 (console)	`minio_data` volume
`sa-minio-init`	`minio/mc:latest`	—	One-shot bucket creator
`sa-gateway`	`nginx:1.27-alpine`	**8080** (public)	—
`sa-auth`	Built from `services/auth`	8000	`./services/auth` (bind mount)
`sa-projection`	Built from `services/projection`	3000	`./services/projection` (bind mount)
`sa-frontend`	Built from `frontend`	5173	`./frontend` (bind mount)

Startup order is enforced via `depends_on` with `condition: service_healthy` checks on PostgreSQL, Redis, and MinIO.

PostgreSQL

- **Image:** `postgres:16-alpine`

- **Database created:** `sa_auth` (the init script at `docker/postgres/init-databases.sh` runs once on volume creation)
- **User:** configured via `POSTGRES_USER` env var (default `saintaugustin`)
- **Stub databases** for future services (`sa_songs`, `sa_playlists`) can be added to the init script when those services are extracted

Connecting directly:

```
make db-shell
# or:
docker compose exec postgres psql -U saintaugustin -d sa_auth
```

Redis

- **Image:** `redis:7-alpine`
- **Used by:** Auth Service (cache + queue driver), Projection Service (session storage)
- **No persistence config** — data is ephemeral and acceptable to lose on restart
- Projection sessions have a 4-hour TTL and clean themselves up automatically

---

MinIO (Object Storage)

- **Image:** `minio/minio:latest`
- **S3 API:** port 9000
- **Web Console:** port 9001 — visit `http://localhost:9001` to browse buckets and files
- **Default bucket:** `saintaugustin` (created by `sa-minio-init` on first start)
- **Public path:** `local/saintaugustin/public` is set to anonymous download — public preview files can go here

Song sheets are stored in the `saintaugustin` bucket. The Auth Service generates short-lived presigned URLs on demand rather than issuing permanent public URLs.

Access credentials are set via `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD` in `.env`.

Nginx Gateway

- **Image:** `nginx:1.27-alpine`
- **Config:** `docker/nginx/default.conf`
- **Public port:** 8080 (configurable via `GATEWAY_HTTP_PORT`)
- **Max upload body:** 50 MB (for song sheet uploads)

The gateway routes all traffic by URL prefix. WebSocket connections (projection and Vite HMR) are handled with `Upgrade` / `Connection` headers and an extended `proxy_read_timeout` (86400 s).

Path routing summary

```
/api/auth/        → auth-service:8000
/api/songs     → auth-service:8000  (currently; placeholder upstreams exist for future sp
/api/songbooks    → auth-service:8000
/api/playlists    → auth-service:8000
/api/share/       → auth-service:8000
/api/sheets       → auth-service:8000
/api/projection/  → projection-service:3000
/ws/projection/   → projection-service:3000 (WebSocket)
/socket.io/       → projection-service:3000 (Socket.IO)
/                 → frontend:5173
```

> **Note:** The Nginx config retains stub upstreams (`song_service`, `playlist_service`, `file_service`, `import_service`) pointing at containers that don't yet exist. These upstreams will return errors if ever matched, but the routing rules that reference them are not currently active.

Volumes

Volume	Purpose
`postgres_data`	PostgreSQL data directory
`redis_data`	Redis RDB snapshot
`minio_data`	MinIO object storage

To completely reset the environment (including all data):

```
docker compose down -v
docker compose up -d
make migrate seed
```

CI / CD

GitHub Actions workflows live in `.github/workflows/`. The pipeline runs on every push and pull request:

- **Projection Service** — builds the Docker image, runs `npm test`
- **Frontend** — builds the Docker image, runs `npm run lint` and `npm test`
- **Auth Service** — builds the Docker image, runs `vendor/bin/pest`

Common Operations

```
# First-time setup
make setup
make migrate seed

# Daily start / stop
make up
make down

# Tail logs
make logs              # all services
make auth-logs         # just auth
make projection-logs   # just projection
make frontend-logs     # just frontend

# Open a database shell
make db-shell

# Drop and recreate all tables + seed
make fresh

# Run tests
make test
make test-auth
make test-projection
```

SaintAugustin – Phase 0 Setup Checklist

Step-by-step instructions for setting up the development environment on your Debian server.

Prerequisites Checklist

- ☐ Debian 12 (bookworm) server accessible via SSH
- ☐ VS Code with Remote - SSH extension
- ☐ `sudo` access on the server
- ☐ GitHub account (for repository + CI)

Step 1: Extract the Scaffold

```
# On your Debian server
cd ~
tar xzf saint-augustin-scaffold.tar.gz
cd saint-augustin
```

Step 2: Install System Dependencies

```
sudo chmod +x scripts/setup-debian.sh
sudo ./scripts/setup-debian.sh
```

This installs: - Docker Engine + Compose v2 (docs) - Node.js 20 LTS (NodeSource) - PHP 8.3 + extensions (sury.org) - Composer 2 (getcomposer.org)

After the script completes, log out and back in (or run `newgrp docker`) so your user can run Docker without `sudo`.

Verify:

```
docker run hello-world          # Docker works without sudo
docker compose version          # Compose v2 installed
node --version                  # v20.x.x
php --version                   # 8.3.x
composer --version              # 2.x.x
```

Step 3: Create Environment Files

```
cd ~/saint-augustin
cp .env.example .env
cp services/auth/.env.example services/auth/.env
```

Edit `.env` and change at minimum: - `POSTGRES_PASSWORD` — pick a dev password - `MINIO_ROOT_PASSWORD` — pick a dev password - `JWT_SECRET` — at least 32 random characters

Step 4: Install Application Dependencies

```
# Auth Service (Laravel)
cd services/auth
composer install
php artisan key:generate   # generates APP_KEY in .env
cd ../..

# Projection Service (NestJS)
cd services/projection
npm i
cd ../..

# Frontend (Vue 3)
cd frontend
npm i
cd ..
```

Step 5: Start Infrastructure

```
# Start PostgreSQL, Redis, MinIO first
docker compose up -d postgres redis minio

# Wait for PostgreSQL to be ready (~10 seconds)
docker compose logs -f postgres   # watch for "ready to accept connections"
# Ctrl+C to exit logs
```

Verify databases were created:

```
docker compose exec postgres psql -U saintaugustin -c '\l'
# Should show: sa_auth, sa_songs, sa_playlists
```

Step 6: Start All Services

```
docker compose up -d
docker compose ps   # all containers should be "running" or "healthy"
```

Step 7: Run Migrations and Seed

```
docker compose exec auth-service php artisan migrate
docker compose exec auth-service php artisan db:seed
```

Step 8: Verify Everything Works

```
# Auth Service health
curl http://localhost:8080/health
# Expected: {"status":"ok"}  (or similar Laravel health response)

# Auth API status
curl http://localhost:8080/api/auth/status
# Expected: {"service":"auth-service","status":"ok","version":"0.1.0"}

# Projection Service health
curl http://localhost:3000/health
# Expected: {"status":"ok","service":"projection-service","timestamp":"..."}
```

Open in browser (replace `your-server` with your server IP or hostname): - **Frontend (direct):** http://your-server:5173 - **Frontend (via gateway):** http://your-server:8080 - **MinIO Console:** http://your-server:9001 (login with MINIO_ROOT_USER/PASSWORD)

If connecting from VS Code Remote SSH, you can forward ports: - VS Code auto-detects open ports and offers to forward them - Or manually: `Ctrl+Shift+P` → "Forward a Port" → enter `8080`, `5173`, `9001`

Step 9: Initialize Git Repository

```
cd ~/saint-augustin
git init
git checkout -b main
git add .
git commit -m "Phase 0: project scaffolding

- Monorepo structure with services/auth, services/projection, frontend
- Docker Compose: PostgreSQL 16, Redis 7, MinIO, Nginx gateway
- Laravel 12 Auth Service skeleton with Octane, Sanctum, health endpoint
- NestJS 10 Projection Service with WebSocket echo gateway (learning spike)
- Vue 3 + Vite + TypeScript frontend with Tailwind CSS, Pinia, vue-router
- GitHub Actions CI pipeline (lint, test, Docker build)
- Debian setup script (Docker, Node 20, PHP 8.3, Composer)
- Makefile with common dev commands"

git checkout -b develop
```

Push to GitHub:

```
# Create the repo on GitHub first, then:
git remote add origin git@github.com:YOUR_USERNAME/saint-augustin.git
git push -u origin main
git push -u origin develop
```

Step 10: Verify CI Pipeline

After pushing, check GitHub → Actions tab. The CI pipeline should: - □ Build Projection Service Docker image - □ Build Frontend Docker image - □ Auth Service tests (commented out until `composer install` runs in CI)

VS Code Recommended Extensions

Install these via the Extensions panel (`Ctrl+Shift+X`):

Extension	ID
Vue - Official	`Vue.volar`
Tailwind CSS IntelliSense	`bradlc.vscode-tailwindcss`
PHP Intelephense	`bmewburn.vscode-intelephense-client`
ESLint	`dbaeumer.vscode-eslint`
Docker	`ms-azuretools.vscode-docker`
EditorConfig	`EditorConfig.EditorConfig`
GitLens	`eamodio.gitlens`

Daily Development Workflow

```
# Start your dev session
cd ~/saint-augustin
docker compose up -d        # or: make up

# Tail logs while working
make logs                   # all services
make auth-logs              # just auth service

# Run tests
make test                   # all tests
make test-auth              # auth only

# Reset database if needed
make fresh                  # migrate:fresh --seed

# Stop when done
docker compose down         # or: make down
```

Troubleshooting

Port already in use:

```
sudo lsof -i :5432    # find what's using the port
# If system PostgreSQL is running:
sudo systemctl stop postgresql
sudo systemctl disable postgresql
```

Docker permission denied:

```
sudo usermod -aG docker $USER
newgrp docker   # or log out and back in
```

Auth Service won't start (APP_KEY missing):

```
cd services/auth
php artisan key:generate
cd ../..
docker compose restart auth-service
```

PostgreSQL init script didn't run:

```
# The init script only runs on first volume creation
docker compose down -v                  # removes volumes
docker compose up -d postgres           # recreates with init
```

Frontend can't reach backend through gateway: Check that all services are healthy:

```
docker compose ps
docker compose logs gateway             # check nginx errors
```